PDD and Profile of Linear Accelerator

Date of experiment: 26 February 2026

Medical Physics Laboratory II

Name: Souvik Das

Roll No: 241126007

National Institute of Science Education and Research
Jatani, Odisha, India

Date of submission: April 25, 2026

Contents

1 Objective

1. To determine the percentage depth dose and profile of the Photon and Electron beam from a medical LINAC
2. Write a python code to find various beam parameters from the data obtained during the measurement.

2 Apparatus

- Medical Linear Accelerator
- 3D Scanner (RFA) with Water Reservoir
- Two Ionization Chamber (Field and reference Ionization chamber)
- Electrometer and Connecting cables.
- Thermometer and Barometer
- Leveling tool (Spirit level)

2.1 Medical Linear Accelerator

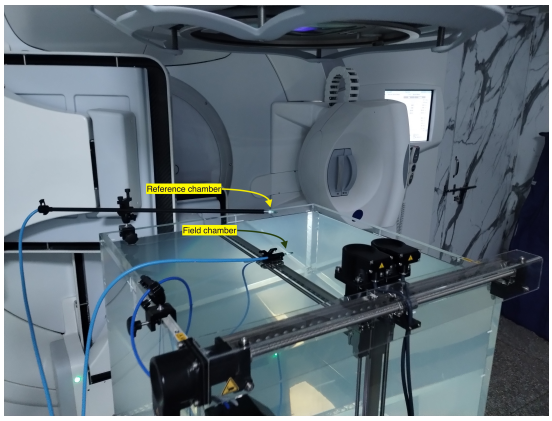
A medical linear accelerator (linac) is a device that uses high-frequency electromagnetic waves to accelerate charged particles, such as electrons, to high energies. These accelerated particles are then used to produce high-energy X-rays or electron beams for radiation therapy. The linac consists of several components, including an electron gun, a waveguide, and a treatment head. The electron gun generates electrons, which are then accelerated through the waveguide by the electromagnetic waves. The treatment head contains various components, such as a target for X-ray production, collimators to shape the beam, and a multileaf collimator for further beam shaping. The linac is a crucial tool in modern radiation therapy, allowing for precise targeting of tumors while minimizing damage to surrounding healthy tissue.

2.2 3D Scanner (RFA) with Water Reservoir

A 3D Scanner with a water reservoir, often referred to as a Radiation Field Analyzer (RFA), is a device used in radiation therapy to measure and analyze the characteristics of radiation beams. The water reservoir is filled with water, which serves as a medium for simulating human tissue. The RFA is equipped with a scanning mechanism that allows it to move through the water, measuring the dose distribution of the radiation beam at various depths and positions. This information is crucial for treatment planning and quality assurance in radiation therapy, as it helps ensure that the prescribed dose is delivered accurately to the target area while minimizing exposure to surrounding healthy tissues. RFA is a large, usually 50 x 50 x 50cm3 water phantom used to perform radiation dosimetry in a clinical setup. The RFA can hold 166 L of water, and it has a railing on which the chamber can be fixed using holders and moved using a control setup from outside. 3 screws are provided at the base of the RFA to level it. The whole mechanical system is such that a very high accuracy of chamber positioning (0.4 mm) can be achieved (as the ion chambers can move in 3 orthogonal directions).The RFA can provide detailed data on the beam's intensity, shape, and uniformity, which are essential for optimizing treatment outcomes.

2.3 Ionization Chambers

Ionization chambers are devices used to measure ionizing radiation. They consist of a gas-filled chamber with two electrodes. When ionizing radiation passes through the chamber, it ionizes the gas, creating positive ions and free electrons. The electrodes collect these charges, producing an electrical signal that is proportional to the amount of radiation. In radiation therapy, two types of ionization chambers are commonly used: the field ionization chamber and the reference ionization chamber. The field ionization chamber is used to measure the dose distribution of the radiation beam as it scans through the water in the RFA. The reference ionization chamber is typically fixed in a corner of the water tank and monitors the beam output in real-time. The signal from the field ionization chamber is normalized against the reference chamber's signal to remove errors caused by fluctuations in the machine's dose rate, ensuring accurate dose measurements for treatment planning and quality assurance.



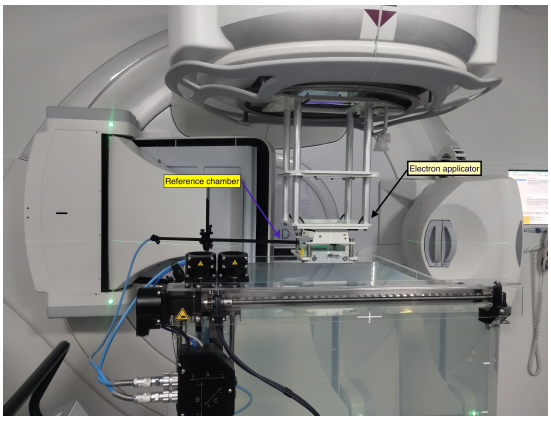


Figure 1: RFA setup for photon and electron beam scanning. Semiflex 3D ionization chamber is used. The cylindrical field Ionization chamber is set 0.6 r_{cyl} above the reference point of the chamber due to its effective measurement point.

Field chambers and reference chambers are used together in PDD (Percentage Depth Dose) and profile scanning to ensure accuracy by correcting for linac output fluctuations, beam energy variations, and mechanical errors during measurements. The field chamber measures the dose distribution as it scans through the water, while the reference chamber continuously monitors the beam output. By normalizing the field chamber's readings against the reference chamber's signal, any variations in the machine's performance are accounted for, ensuring that the measured dose distribution accurately reflects the true characteristics of the radiation beam. This is essential for reliable treatment planning and quality assurance in radiation therapy.

2.4 Electrometer and Connecting Cables

An electrometer is a sensitive electronic device used to measure electric charge or current. In the context of radiation therapy, electrometers are used to measure the electrical signals produced by ionization chambers when they detect ionizing radiation. The electrometer amplifies and converts these signals into readable values, which can then be used to calculate the dose of radiation being delivered. Connecting cables are essential for linking the ionization chambers to the electrometer, allowing for the transmission of electrical signals. These cables must be of high quality to ensure accurate signal transmission without interference or loss, which is crucial for obtaining precise measurements in radiation therapy.

2.5 Thermometer and Barometer

A thermometer is an instrument used to measure temperature, while a barometer is used to measure atmospheric pressure. In radiation therapy, both of these instruments are important for ensuring accurate dose measurements. The temperature and pressure of the environment can affect the density of the air in the ionization chambers, which in turn can influence the readings obtained from these chambers. By monitoring and accounting for changes in temperature and pressure, clinicians can apply necessary corrections to the dose measurements, ensuring that the prescribed dose is delivered accurately to the patient. This is particularly important in radiation therapy, where precise dosing is critical for effective treatment and minimizing side effects.

2.6 Leveling Tool (Spirit Level)

A leveling tool, such as a spirit level, is used to ensure that the equipment and setup in radiation therapy are properly aligned and level. Proper leveling is crucial for accurate dose delivery, as any tilt or misalignment can lead to uneven dose distribution and potentially harm healthy tissues while underdosing the target area.

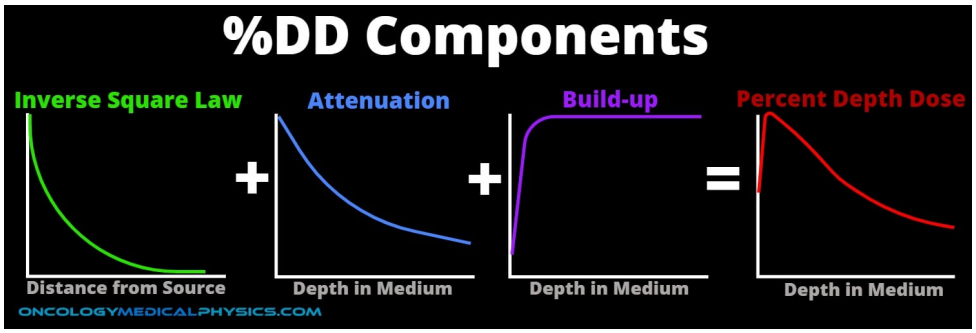


Figure 2: Percentage Depth Dose (PDD) curve showing the variation of absorbed dose with depth for a photon beam. The curve illustrates the build-up region, the depth of maximum dose (d_{max}), and the subsequent decrease in dose with increasing depth due to both inverse square law and attenuation.

3 Theory

As the beam enters the patient, the absorbed dose at a point varies with depth. This variation depends on many conditions: Beam energy, Field size, distance from the source, and Beam collimation system. Thus, calculating the dose in a patient involves considering these parameters and others as they affect depth dose distribution. Hence, the dose calculation system needs to establish percentage depth dose variation along the central axis of the beam.

The Dose increases with depth as we move into depth from the surface up to a maximum value and then decreases with depth. The dose region between the surface and the depth (d_{max}) of the maximum dose is called the dose build-up region. This increase in dose deposition is due to the increased range of secondary electrons released in the tissue. The larger the photon energy, the larger the range of secondary electrons, and consequently, the larger the depth of dose maximum (d_{max}).

3.1 Beam Parameters

3.1.1 Percentage Depth Dose(PDD)

The Percentage depth dose (PDD) is defined as the quotient, expressed as a percentage, of the absorbed dose at any depth d to the absorbed dose at a reference depth d_0, along the central axis of the beam for a fixed source to surface distance. Mathematically, it can be expressed as:

$$PDD(d) = \frac{D(d)}{D(d_0)} \times 100\% \tag{1}$$

Where $D(d)$ is the absorbed dose at depth d and $D(d_0)$ is the absorbed dose at the reference depth d_0. The reference depth d_0 is typically chosen as the depth of maximum dose (d_{max}) for the given beam energy and field size.

3.1.2 Surface Dose

The surface dose is the absorbed dose at the surface of the patient. It is an important parameter in radiation therapy as it can affect the skin and superficial tissues. The surface dose is influenced by factors such as beam energy, field size, and the presence of any bolus material. Generally, for photon beams, the surface dose is lower than the dose at depth due to the build-up effect, where secondary electrons generated by photon interactions deposit their energy at some distance from the point of interaction.

3.1.3 Practical Range

In the case of an electron beam, the depth dose at the end remains constant after a certain depth due to the bremsstrahlung production. So, the quantity 'Practical range' is defined as the depth at which is the depth of the point where the tangent to the descending linear portion of the curve (at the point of inflection) intersects the extrapolated background.

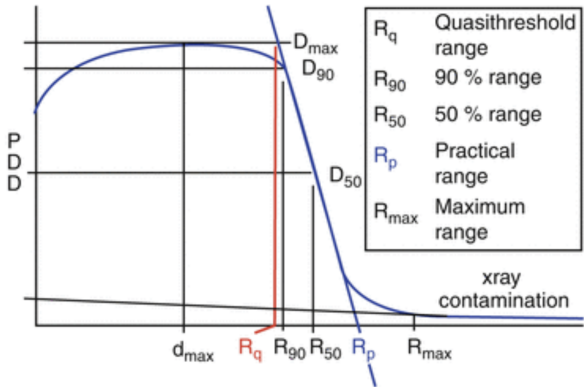


Figure 3: PDD curve for an electron beam showing the practical range, which is the depth at which the dose falls to zero. The curve illustrates the rapid dose fall-off after the practical range due to the limited penetration of electrons in tissue.

3.1.4 Beam Flatness

The beam flatness for the **photon beam** is assessed by finding the maximum D_{max} and minimum D_{min} dose point values on the beam profile within the central 80% of the beam width and then using the following relationship. Standard linac specifications generally require that F be less than 3% when measured in a water phantom at a depth of 10 cm and an SSD of 100 cm for the largest field size available (usually 40 × 40 cm^2).

$$F = \frac{D_{max} - D_{min}}{D_{max} + D_{min}} \times 100\% \tag{2}$$

Flatness for **electron** beam along major axes at the depth of dose maximum as the separation between 90% and 50% dose points on either side of the beam profile for 10×10 cm^2 applicator and maximum applicator size. The tolerance value is 10 mm.

3.1.5 Beam Symmetry

The beam symmetry for the **photon** is usually determined at z_{max}, which represents the most sensitive depth for assessment of this beam uniformity parameter. A typical symmetry specification is that any two dose points on a beam profile, equidistant from the central axis point, are within 2% of each other. Alternately, areas under the z_{max} beam profile on each side (left and right) of the central axis extending to the 50% dose level (normalized to 100% at the central axis point) are determined, and symmetry is then calculated from

$$Symmetry = \frac{Area_{left} - Area_{right}}{Area_{left} + Area_{right}} \times 100\% \tag{3}$$

Symmetry for **electron** beam along major axes at depth of dose maxima in SSD setup (maximum ratio of absorbed doses at symmetrical points from central beam axis and more than 1 cm inside the 90isodose contour).

3.1.6 Penumbra

The photon penumbra is typically defined as the distance between the 80% and 20% dose points on a transverse beam profile measured 10 cm deep in a water phantom. The electron penumbra is usually defined as the distance between the 80% and 20% dose points along a major axis at a given depth. The IEC defines this depth as one-half of the depth of the 80% dose on the central axis.

3.2 FFF Beam Profile Parameters (AERB Guidelines)

As per the AERB guidance document for un-flat (FFF) beams, acceptance and QA measurements are to be established as baseline values and then used for periodic constancy checks [1]. The guideline specifies the following points:

1. Clinical use of FFF mode should be through TPS and Record & Verify networked workflow; manual planning/calculation is not to be used.

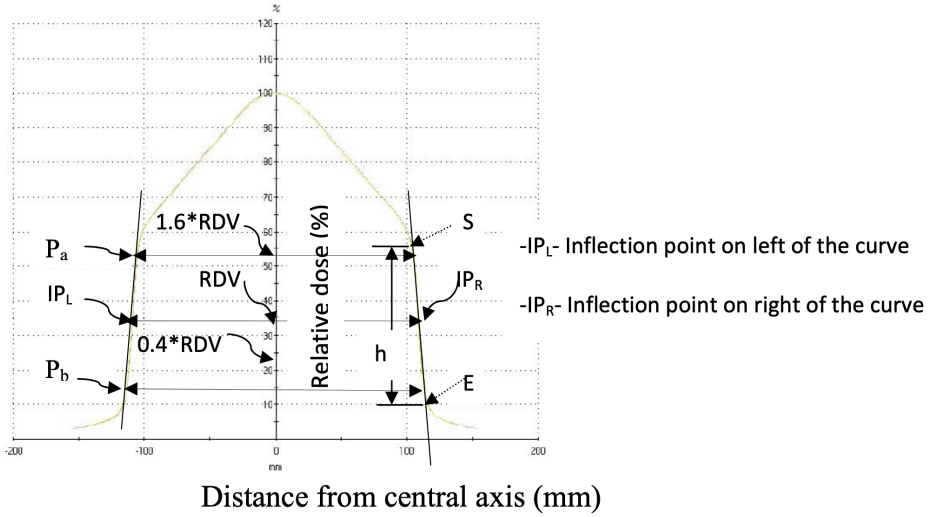


Figure 4: Schematic diagram for determining inflection point and penumbra. [1]

2. For report submission and baseline generation, measurements include: nominal energy with measured $TPR_{20/10}$ (10×10 cm^2), surface dose (10×10 and 20×20 cm^2), OAR at ±3 cm (10×10 cm^2 at 10 cm depth), depth-dose profiles (5×5, 10×10, 20×20 cm^2), and beam profiles (20×20 cm^2, depth 10 cm, SAD setup).

3. For accelerators with FFF maximum field size less than 10×10 cm^2, profile parameters are evaluated using flat-beam methods; where flatness exceeds $\pm3\%$, un-flat beam criteria are adopted.

3.2.1 Inflection Point (AERB Practical Definition)

AERB states that the inflection point should follow its mathematical definition. For practical QA identification on each side of the profile, it may be approximated as the midpoint of the high-gradient (sharply descending) region. The procedure is: identify the start point (S) and end point (E) of the high-gradient region, determine the corresponding profile height span h, and mark the inflection point at $h/2$ from S (or equivalently from E) [1].

For profile constancy, the lateral separation between the left and right inflection points (IPL and IPR) along major axes is recorded as a baseline quantity and compared in periodic QA [1].

3.2.2 Degree of Unflatness (AERB Method)

To quantify degree of unflatness, AERB recommends recording the lateral distances from the central axis to the 90%, 75%, and 60% relative-dose points on both sides of the profile along major axes for all available FFF energies (parameters typically tabulated as X90, X75, and X60) [1].

3.2.3 Related Penumbra Criterion for FFF Profile

For FFF penumbra determination, the dose value at the inflection point is taken as the reference dose value (RDV). Points corresponding to $1.6\times$RDV and $0.4\times$RDV are identified on each side, and their lateral separation is reported as penumbra along the major axes [1].

The guideline indicates these profile-related values should be maintained as baseline data; periodic QA is monthly, and tolerances are assessed with respect to baseline values (as indicated in the AERB table for un-flat beam profile QA) [1].

3.3 Conversion of Depth Ionization to Absorbed Dose

Depth-dose and isodose distributions can be measured with ionization chambers, diodes, or films; however, for electron-beam depth measurements, plane-parallel chambers are generally preferred for relative dosimetry, while cylindrical chambers are also used in practice. When a chamber is used, the

measured depth-ionization curve is not directly the depth-dose curve. It must be converted by applying stopping-power and chamber-replacement corrections [2].

For electron beams measured in water, the conversion from percent depth ionization to percent depth dose can be written as

$$\%DD_w(d) = \%DI_w(d) \times \frac{\left[(\overline{L}/\rho)^w_{air} \times P_{repl}\right]_d}{\left[(\overline{L}/\rho)^w_{air} \times P_{repl}\right]_{d_{max}}}. \tag{4}$$

Here, $(\overline{L}/\rho)^w_{air}$ is the restricted mass collision stopping-power ratio of water to air, and P_{repl} is the chamber replacement factor. The factor P_{repl} includes: (i) in-scatter effect, (ii) obliquity effect, and (iii) effective point-of-measurement displacement. The first two are usually grouped as fluence correction, while the third is treated as gradient correction. In practical scanning, this means the ionization curve is shifted and scaled through these factors before reporting PDD [2].

4 Calculation and Observation

4.1 Observed PDD Curves

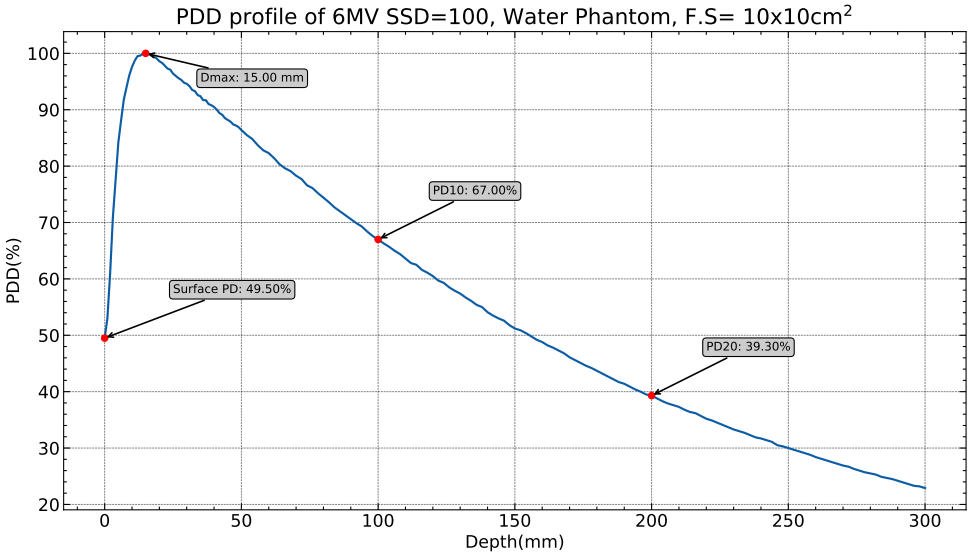


Figure 5: Percentage depth dose for 6 MV photon beam.

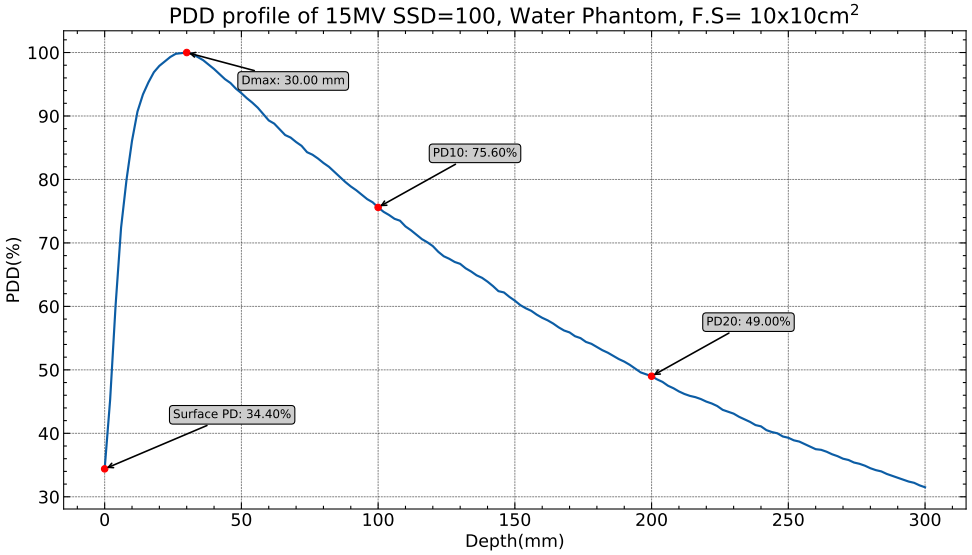


Figure 6: Percentage depth dose for 15 MV photon beam.

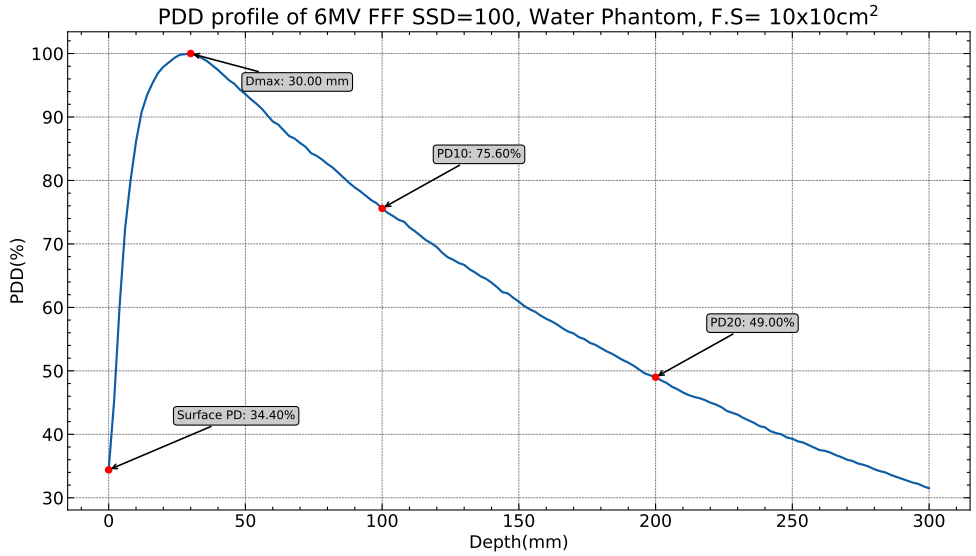


Figure 7: Percentage depth dose for 6 MV FFF photon beam.

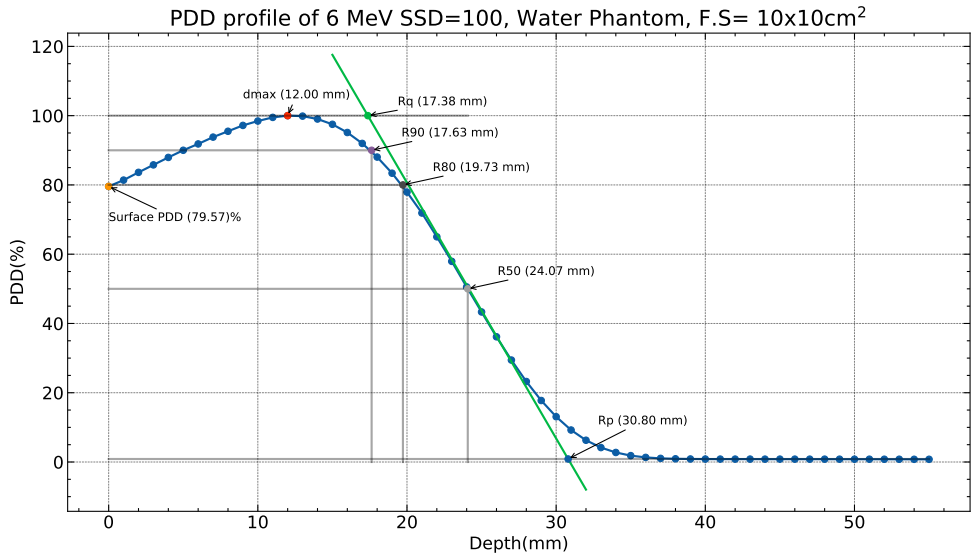


Figure 8: Percentage depth dose for 6 MeV electron beam.

4.2 Observed Beam Profiles

4.2.1 6 MV Photon (10 x 10 cm2)

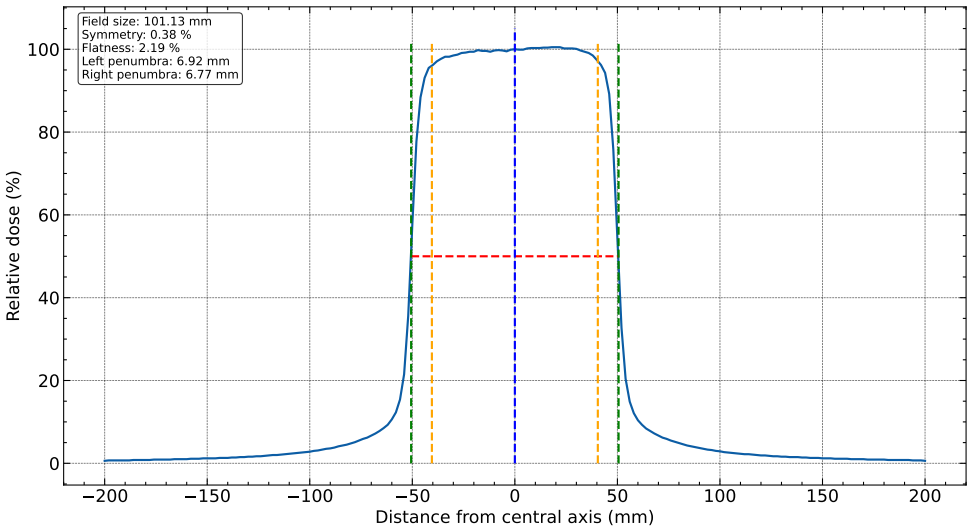


Figure 9: Inline beam profile for 6 MV photon beam at 10 × 10 cm^2.

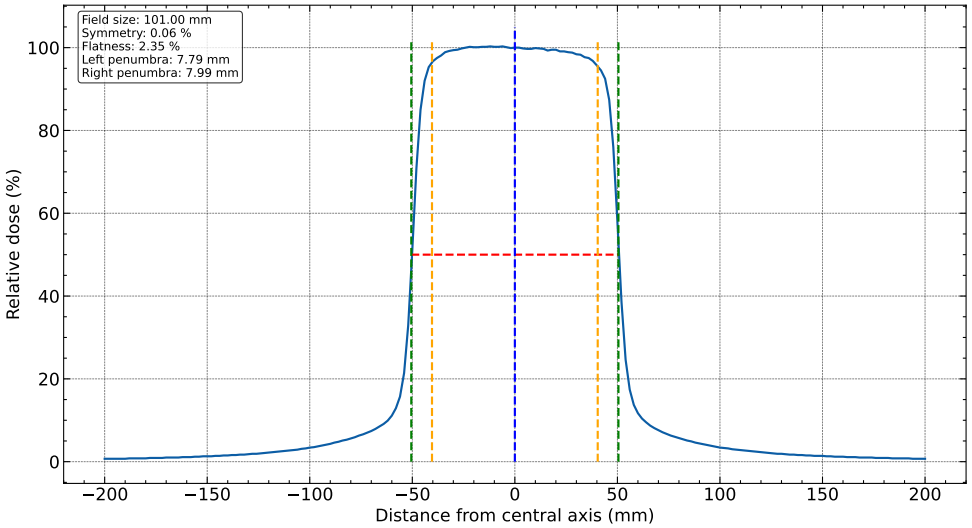


Figure 10: Crossline beam profile for 6 MV photon beam at 10 × 10 cm^2.

4.2.2 6 MV Photon (35 x 35 cm2)

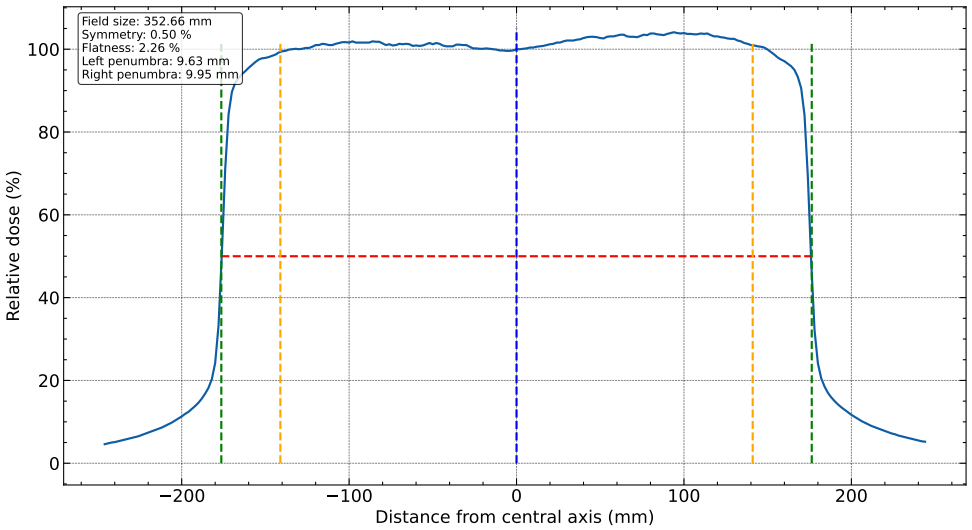


Figure 11: Inline beam profile for 6 MV photon beam at 35 × 35 cm^2.

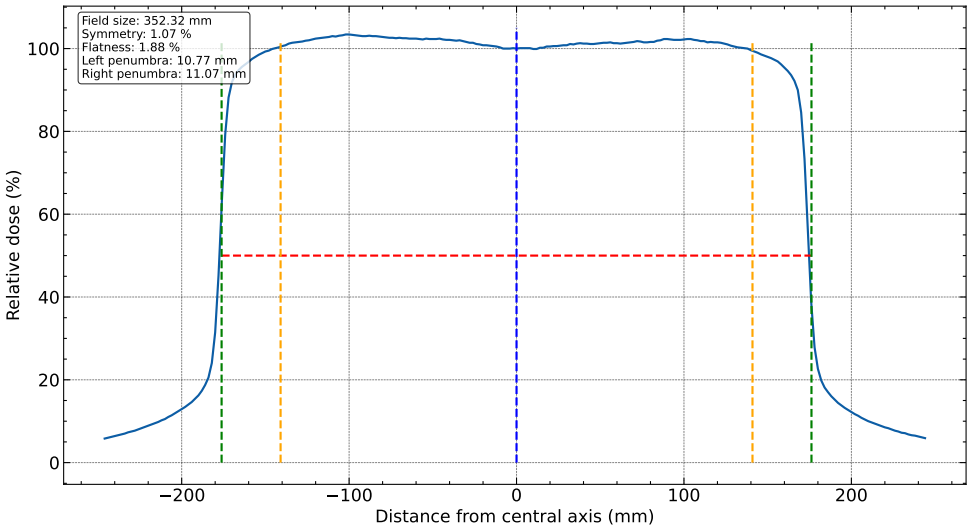


Figure 12: Crossline beam profile for 6 MV photon beam at 35 × 35 cm^2.

4.2.3 6 MV FFF Photon (20 x 20 cm2)

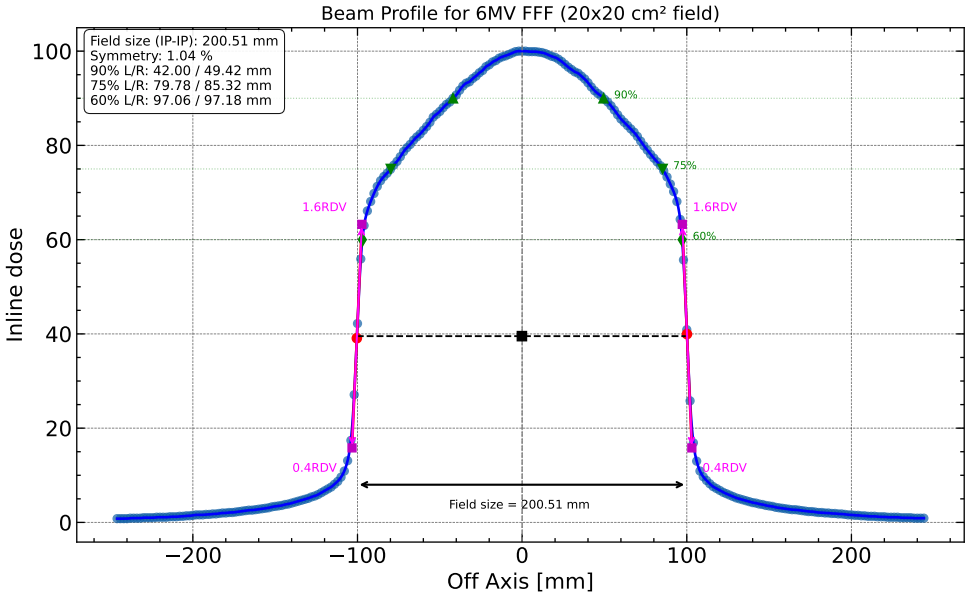


Figure 13: Inline beam profile for 6 MV FFF photon beam at 20×20 cm^2.

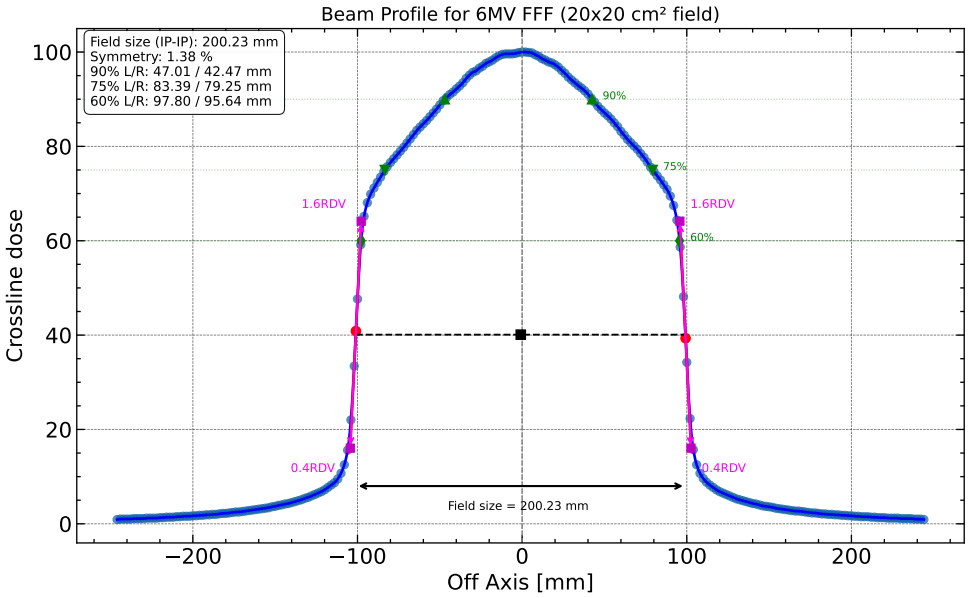


Figure 14: Crossline beam profile for 6 MV FFF photon beam at 20×20 cm^2.

4.2.4 6 MeV Electron (10 x 10 cm2)

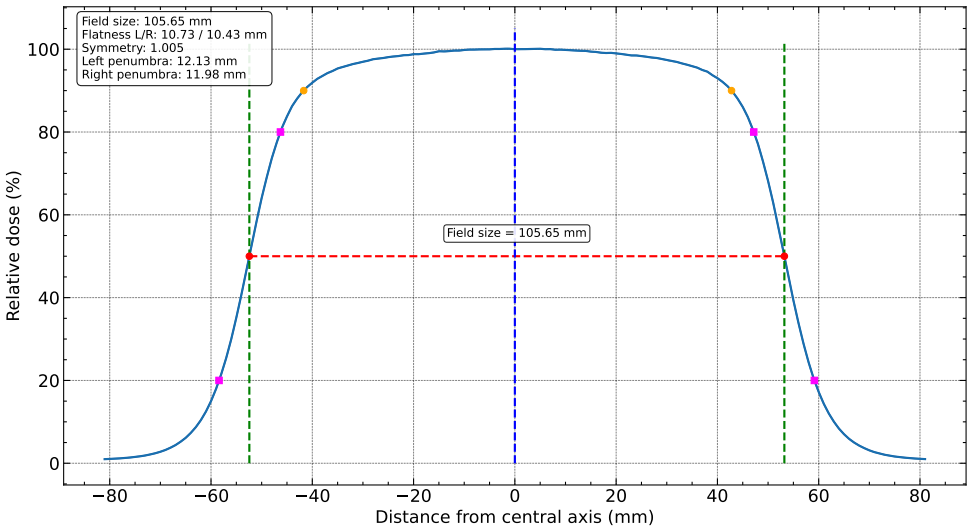


Figure 15: Inline beam profile for 6 MeV electron beam at 10 × 10 cm^2.

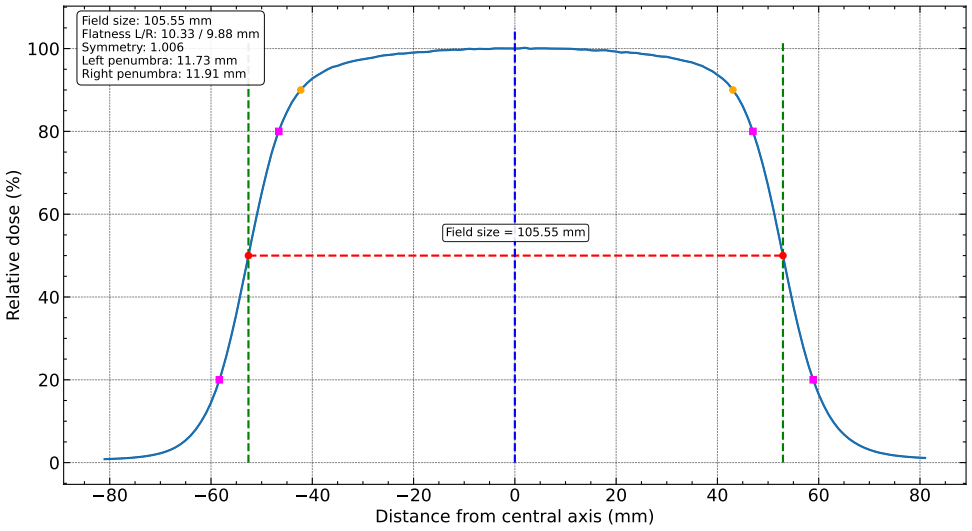


Figure 16: Crossline beam profile for 6 MeV electron beam at 10 × 10 cm^2.

4.3 Python Programs Used for Analysis

Listing 1: Python program for photon PDD analysis.

```
# Importing some useful library
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scienceplots
plt.style.use(['science', 'notebook', 'grid'])

# Data Extraction from the xlsx file
df = pd.read_excel("PDD Group 2.xlsx", sheet_name=0)
x = np.array(df["Depth[mm]"])
y = np.array(df["Depth dose"])

# Percentage depth dose
PDD = y/max(y)*100

# To find the surface PD(Percentage dose), dmax, D10(Dose at 10 cm depth), D20
# variable initilization
SPD = 0 # surface PD
dmax = 0; x90 = 0; x50 = 0; PDD90 = 0; PDD50 = 0
for i in range(len(PDD)):
    if x[i] == 0:
        SPD = PDD[i]

    if PDD[i] == 100:
        dmax = x[i]


# To find the surface PD(Percentage dose), dmax, D10(Dose at 10 cm depth), D20
# variable initilization
SPD = 0 # surface PD
dmax = 0; D10 = 0;D20 = 0
for i in range(len(PDD)):
    if PDD[i] == 100:
        dmax = x[i]
    if x[i] == 0:
        SPD = PDD[i]
    if x[i] == 100: # 100 mm
        D10 = PDD[i]
    if x[i] == 200: # 200 mm
        D20 = PDD[i]



#Plotting
plt.figure(figsize=(15, 8))
plt.title(r"PDD profile of 6MV FFF SSD=100, Water Phantom, F.S= 10x10cm$^2$",
    fontsize=20)
plt.plot(x, PDD)
bbox = dict(boxstyle="round", fc="0.8")
plt.scatter([0, dmax, 100, 200], [SPD, 100, D10, D20], color='red', zorder=3)
plt.annotate(f"Surface PD: {SPD:.2f}%", xy=(0, SPD), xytext=(25, SPD + 8),bbox=
    bbox,
             arrowprops=dict(arrowstyle="->", lw=1.5), fontsize=11)
plt.annotate(f"Dmax: {dmax:.2f} mm", xy=(dmax, 100), xytext=(dmax + 20, 95),bbox
    =bbox,
             arrowprops=dict(arrowstyle="->", lw=1.5), fontsize=11)
plt.annotate(f"PD10: {D10:.2f}%", xy=(100, D10), xytext=(120, D10 + 8),bbox=bbox
    ,
             arrowprops=dict(arrowstyle="->", lw=1.5), fontsize=11)
```

```
plt.annotate(f"PD20: {D20:.2f}%", xy=(200, D20), xytext=(220, D20 + 8),bbox=bbox
    ,
              arrowprops=dict(arrowstyle="->", lw=1.5), fontsize=11)
plt.ylabel("PDD(%)")
plt.xlabel("Depth(mm)")
#Save the figure in pdf format
plt.savefig("Figures/PDD/PDD_6MVFFFPhoton.pdf", format='pdf')
plt.show()
```

Listing 2: Python program for electron PDD analysis.

```
# Importing some useful library
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scienceplots
plt.style.use(['science', 'notebook', 'grid'])

# Data Extraction from the xlsx file
df = pd.read_excel("PDD Group 2.xlsx", sheet_name=3)
x = np.array(df["Depth[mm]"])
y = np.array(df["Depth dose"])

# Percentage depth dose
PDD = y/max(y)*100

# To find the surface PD(Percentage dose), dmax, D10(Dose at 10 cm depth), D20
# variable initilization
SPD = 0 # surface PD
dmax = 0; x90 = 0; x50 = 0; PDD90 = 0; PDD50 = 0; PDDm = 0; xm = 0
for i in range(len(PDD)):
    if x[i] == 0:
        SPD = PDD[i]

    if PDD[i] == 100:
        dmax = x[i]

    if PDD[i] > 90:
        x90 = x[i] + ((x[i-1] - x[i])/(PDD[i-1] - PDD[i]))*(90 - PDD[i]) #
            Straight line interpolation
    if PDD[i] > 80:
        x80 = x[i] + ((x[i-1] - x[i])/(PDD[i-1] - PDD[i]))*(80 - PDD[i]) #
            Straight line interpolation
    if PDD[i] > 50:
        x50 = x[i] + ((x[i-1] - x[i])/(PDD[i-1] - PDD[i]))*(50 - PDD[i]) #
            Straight line interpolation
        PDD50 = PDD[i]
        m = (PDD[i] - PDD[i-1])/(x[i] - x[i-1])

    if abs(PDD[i]- PDD[i-1]) > 0.05:
        xm = x[i]; PDDm = PDD[i]


xx = np.linspace(15, 32, 1000)
yy = PDD50 + m*(xx - x50)
c1 = PDD50 - m*x50
c2 = PDDm
xp = (c2 - c1)/m
xq = (100 - c1)/m



#Plotting
plt.figure(figsize=(15, 8))
```

```
plt.title(r"PDD profile of 6 MeV SSD=100, Water Phantom, F.S= 10x10cm$^2$",
    fontsize=20)
plt.plot(x, PDD, '-o')
plt.plot(xx, yy)

plt.plot(x[0], PDD[0], "o") #Surface PDD
plt.plot(dmax, 100, "o") # Dmax
plt.plot([0, x50], [100, 100], "k", alpha=0.35)

plt.plot([0, x90], [90, 90], "k", alpha=0.35)
plt.plot([x90, x90], [0, 90], "k", alpha=0.35)
plt.plot([x90], [90], "o") #R90


plt.plot([0, x80], [80, 80], "k", alpha=0.35)
plt.plot([x80, x80], [0, 80], "k", alpha=0.35)
plt.plot([x80], [80], "o") #R80

plt.plot([0, x50], [50, 50], "k", alpha=0.35)
plt.plot([x50, x50], [0, 50], "k", alpha=0.35)
plt.plot([x50], [50], "o") #R50

plt.plot([0, xm, x[-1]], [PDDm, PDDm, PDDm], "k", alpha=0.35)
plt.plot([xp], [PDDm], "o") #Rp

plt.plot([xq], [100], "o") #Rq

surface_label = f"Surface PDD ({PDD[0]:.2f})%"
dmax_label = f"dmax ({dmax:.2f} mm)"
x90_label = f"R90 ({x90:.2f} mm)"
x80_label = f"R80 ({x80:.2f} mm)"
x50_label = f"R50 ({x50:.2f} mm)"
xp_label = f"Rp ({xp:.2f} mm)"
xq_label = f"Rq ({xq:.2f} mm)"

plt.annotate(surface_label, xy=(x[0], PDD[0]), xytext=(x[0] , PDD[0] -10),
             arrowprops=dict(arrowstyle="->", lw=1.0), fontsize=11)
plt.annotate(dmax_label, xy=(dmax, 100), xytext=(dmax - 3.0, 105.0),
             arrowprops=dict(arrowstyle="->", lw=1.0), fontsize=11)
plt.annotate(x90_label, xy=(x90, 90), xytext=(x90 + 2.0, 94.0),
             arrowprops=dict(arrowstyle="->", lw=1.0), fontsize=11)
plt.annotate(x80_label, xy=(x80, 80), xytext=(x80 + 2.0, 84.0),
             arrowprops=dict(arrowstyle="->", lw=1.0), fontsize=11)
plt.annotate(x50_label, xy=(x50, 50), xytext=(x50 + 2.0, 54.0),
             arrowprops=dict(arrowstyle="->", lw=1.0), fontsize=11)
plt.annotate(xp_label, xy=(xp, PDDm), xytext=(xp + 2.0, PDDm + 10.0),
             arrowprops=dict(arrowstyle="->", lw=1.0), fontsize=11)
plt.annotate(xq_label, xy=(xq, 100), xytext=(xq + 2.0, 103.0),
             arrowprops=dict(arrowstyle="->", lw=1.0), fontsize=11)

plt.ylabel("PDD(%)")
plt.xlabel("Depth(mm)")
plt.savefig("Figures/PDD/PDD_6MeV_Electron.pdf", dpi=300, bbox_inches='tight')
plt.show()
```

Listing 3: Python program for photon flattened-beam profile analysis.

```
# Importing some useful library
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import scienceplots
plt.style.use(['science', 'notebook', 'grid'])
```

```

# Data Extraction from the xlsx file
df = pd.read_excel("PROFILE Group 2.xlsx", sheet_name=1)
x = np.array(df["Off Axis [mm]"])
y = np.array(df["Inline dose"])
y = y / y[x==0] * 100


#Data analysis
x50L = 0; x50R = 0; x80L = 0; x80R = 0; I80R = []; I80L = []
n = int(len(x)/2)
for i in range(len(x)):
    #Geometric field size
    if y[i] > 50:
        x50L = x[i] + ((x[i] - x[i-1])/(y[i] - y[i-1]))*(50 - y[i])
    if y[-i] > 50:
        x50R = x[-i] + ((x[-i] - x[-i-1])/(y[-i] - y[-i-1]))*(50 - y[-i])
        break

FieldWidth = x50R - x50L

Dmax = 0; Dmin = 100
for i in range(len(x)):
    # Flatness
    if abs(x[i]) <= 0.4*FieldWidth:
        if Dmax < y[i]:
            Dmax = y[i]
        if Dmin > y[i]:
            Dmin = y[i]

# Pnumbra lateral distance between 20% and 80% dose
def find_crossing_x(side_x, side_y, level):
    """Return x-position where profile crosses a given dose level."""
    for i in range(len(side_y) - 1):
        y1, y2 = side_y[i], side_y[i + 1]
        if (y1 >= level and y2 <= level) or (y1 <= level and y2 >= level):
            if y2 == y1:
                return side_x[i]
            return side_x[i] + (level - y1) * (side_x[i + 1] - side_x[i]) / (y2
                - y1)
    raise ValueError(f"No crossing found for {level}% dose level.")


# Split profile around central axis (x ~= 0)
center_idx = np.argmin(np.abs(x))

# Left side from center outward (toward negative x)
left_x = x[:center_idx + 1][::-1]
left_y = y[:center_idx + 1][::-1]

# Right side from center outward (toward positive x)
right_x = x[center_idx:]
right_y = y[center_idx:]

# Left and right 80% and 20% positions
x80_left = find_crossing_x(left_x, left_y, 80)
x20_left = find_crossing_x(left_x, left_y, 20)
x80_right = find_crossing_x(right_x, right_y, 80)
x20_right = find_crossing_x(right_x, right_y, 20)

# Penumbra width = lateral distance between 80% and 20% dose
left_penumbra = abs(x20_left - x80_left)
right_penumbra = abs(x20_right - x80_right)
```

```

y50 = y[y > 50]
n = int(len(y50)/2); h = abs(x[1] - x[0])
AreaL = 0.5*h*(y50[0] + y50[n] + 2*sum(y50[1:n]))
AreaR = 0.5*h*(y50[n+1] + y50[-1] + 2*sum(y50[n+1:-1]))
Symmetry = abs(AreaL - AreaR)/(AreaL + AreaR) * 100


flatness = (Dmax - Dmin) / (Dmax + Dmin) * 100
print("Flatness: ", flatness)
print("Symmetry: ", Symmetry)
print(f"Left penumbra (80%-20%): {left_penumbra:.3f} mm")
print(f"Right penumbra (80%-20%): {right_penumbra:.3f} mm")




#Plotting
fig, ax = plt.subplots(figsize=(15, 8))
ax.plot(x, y, label="Beam Profile")
plt.plot([-FieldWidth/2, FieldWidth/2], [50, 50], color='red', linestyle='--',
    label='50% Line')
plt.plot([-FieldWidth/2, -FieldWidth/2], [0, 102], color='green', linestyle='--'
    )
plt.plot([FieldWidth/2, FieldWidth/2], [0, 102], color='green', linestyle='--')
plt.plot([0, 0], [0, 105], color='blue', linestyle='--')
plt.plot([0.4*FieldWidth, 0.4*FieldWidth], [0, 102], color='orange', linestyle='
    --')
plt.plot([-0.4*FieldWidth, -0.4*FieldWidth], [0, 102], color='orange', linestyle
    ='--')
info_text = (
    f"Field size: {FieldWidth:.2f} mm\n"
    f"Symmetry: {Symmetry:.2f} %\n"
    f"Flatness: {flatness:.2f} %\n"
    f"Left penumbra: {left_penumbra:.2f} mm\n"
    f"Right penumbra: {right_penumbra:.2f} mm"
)

ax.text(0.02, 0.98, info_text, transform=ax.transAxes, va='top', ha='left',
        fontsize=11, bbox=dict(boxstyle='round', facecolor='white', alpha=0.8,
            edgecolor='black'))
ax.set_xlabel("Distance from central axis (mm)")
ax.set_ylabel("Relative dose (%)")
plt.savefig("Figures/Profile/6 MV photon(35 by 35)/InlineProfile.pdf", dpi=300,
    bbox_inches='tight')
plt.show()
```

Listing 4: Python program for FFF beam profile analysis.

```
# Importing some useful library
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scienceplots
from scipy.interpolate import CubicSpline
plt.style.use(['science', 'notebook', 'grid'])

# Data Extraction from the xlsx file
df = pd.read_excel("PROFILE Group 2.xlsx", sheet_name=2)
x = np.array(df["Off Axis [mm]"])
y = np.array(df["Crossline dose"])
center_idx_raw = np.argmin(np.abs(x))
```

```
y = y / y[center_idx_raw] * 100

# Sort data by x to satisfy spline requirements.
sort_idx = np.argsort(x)
x = x[sort_idx]
y = y[sort_idx]

# Build a cubic spline interpolation of the beam profile.
spline = CubicSpline(x, y)

# Dense sampling for smooth curve display and derivative analysis.
x_dense = np.linspace(x.min(), x.max(), 3000)
y_dense = spline(x_dense)
d2y_dense = spline(x_dense, 2)

# Find inflection candidates where second derivative changes sign.
sign_change_idx = np.where(np.diff(np.sign(d2y_dense)) != 0)[0]
inflection_x = []
for idx in sign_change_idx:
    x1, x2 = x_dense[idx], x_dense[idx + 1]
    y1, y2 = d2y_dense[idx], d2y_dense[idx + 1]
    if y2 != y1:
        x_root = x1 - y1 * (x2 - x1) / (y2 - y1)
        inflection_x.append(x_root)

if len(inflection_x) == 0:
    raise RuntimeError("No inflection point found in the interpolated cubic curve.")

inflection_x = np.array(inflection_x)
inflection_slope = spline(inflection_x, 1)

left_mask = inflection_x < 0
right_mask = inflection_x > 0
if not np.any(left_mask) or not np.any(right_mask):
    raise RuntimeError("Could not find inflection points on both sides of central axis.")

# Left side: steepest descent outward corresponds to largest positive dy/dx.
left_idx_local = np.argmax(inflection_slope[left_mask])
# Right side: steepest descent outward corresponds to most negative dy/dx.
right_idx_local = np.argmin(inflection_slope[right_mask])

x_inflect_left = inflection_x[left_mask][left_idx_local]
y_inflect_left = spline(x_inflect_left)
m_left = inflection_slope[left_mask][left_idx_local]

x_inflect_right = inflection_x[right_mask][right_idx_local]
y_inflect_right = spline(x_inflect_right)
m_right = inflection_slope[right_mask][right_idx_local]

# RDV for each side is dose value at inflection point.
rdv_left = y_inflect_left
rdv_right = y_inflect_right
rdv_avg = 0.5 * (rdv_left + rdv_right)

def tangent_x_at_y(x0, y0, m, y_level):
    if np.isclose(m, 0.0):
        raise RuntimeError("Tangent slope is too small for robust penumbra estimation.")
    return x0 + (y_level - y0) / m

def side_penumbra_from_rdv(x0, y0, m, rdv):
```

```
    y_upper = 1.6 * rdv
    y_lower = 0.4 * rdv
    x_upper = tangent_x_at_y(x0, y0, m, y_upper)
    x_lower = tangent_x_at_y(x0, y0, m, y_lower)
    return x_upper, x_lower, abs(x_upper - x_lower), y_upper, y_lower

xu_l, xl_l, penumbra_left, yu_l, yl_l = side_penumbra_from_rdv(x_inflect_left,
    y_inflect_left, m_left, rdv_avg)
xu_r, xl_r, penumbra_right, yu_r, yl_r = side_penumbra_from_rdv(x_inflect_right,
     y_inflect_right, m_right, rdv_avg)

def crossing_from_center(level, side):
    center_idx = np.argmin(np.abs(x_dense))
    if side == "left":
        xs = x_dense[:center_idx + 1][::-1]
        ys = y_dense[:center_idx + 1][::-1]
    else:
        xs = x_dense[center_idx:]
        ys = y_dense[center_idx:]

    for i in range(len(ys) - 1):
        y1, y2 = ys[i], ys[i + 1]
        if (y1 >= level and y2 <= level) or (y1 <= level and y2 >= level):
            if np.isclose(y2, y1):
                return xs[i]
            return xs[i] + (level - y1) * (xs[i + 1] - xs[i]) / (y2 - y1)
    raise RuntimeError(f"No crossing found for {level}% on {side} side.")

levels = [90, 75, 60]
aerb_left = {lev: crossing_from_center(lev, "left") for lev in levels}
aerb_right = {lev: crossing_from_center(lev, "right") for lev in levels}

# Field size for FFF: lateral separation between inflection points.
field_size_inflection = x_inflect_right - x_inflect_left

# Symmetry evaluated similarly to flat-beam area comparison method
# over region bounded by left/right inflection points.
left_region = (x_dense >= x_inflect_left) & (x_dense <= 0)
right_region = (x_dense >= 0) & (x_dense <= x_inflect_right)
area_left = np.trapezoid(y_dense[left_region], x_dense[left_region])
area_right = np.trapezoid(y_dense[right_region], x_dense[right_region])
symmetry = abs(area_left - area_right) / (area_left + area_right) * 100

plt.figure(figsize=(12, 7))
plt.plot(x, y, "o", alpha=0.7)
plt.plot(x_dense, y_dense, "b-", linewidth=2)

# CAX line for distance measurements.
plt.axvline(0, color="gray", linestyle="--", linewidth=1)

# Inflection points used to define the averaged RDV.
plt.plot([x_inflect_left, x_inflect_right], [y_inflect_left, y_inflect_right], "
    ro", markersize=7)

# Averaged RDV reference.
x_rdv_mid = 0.5 * (x_inflect_left + x_inflect_right)
plt.plot([x_inflect_left, x_inflect_right], [rdv_avg, rdv_avg], color="black",
    linestyle="--", linewidth=1.4)
plt.plot(x_rdv_mid, rdv_avg, "ks", markersize=7)

# Tangent lines at inflection points, clipped to RDV construction range.
plt.plot([xl_l, xu_l], [yl_l, yu_l], "r-", linewidth=2)
plt.plot([xl_r, xu_r], [yl_r, yu_r], "r-", linewidth=2)
```

```
133
134 # 1.6*RDV and 0.4*RDV points on tangents.
135 plt.plot([xu_l, xl_l], [yu_l, yl_l], "ms", markersize=6)
136 plt.plot([xu_r, xl_r], [yu_r, yl_r], "ms", markersize=6)
137
138 # AERB points (90/75/60%) on both sides.
139 for lev, marker in zip(levels, ["^", "v", "d"]):
140     plt.plot(aerb_left[lev], lev, marker=marker, color="green", markersize=7)
141     plt.plot(aerb_right[lev], lev, marker=marker, color="green", markersize=7)
142
143 # Direct annotations instead of legend.
144 plt.annotate("1.6RDV", xy=(xu_l, yu_l), xytext=(-45, 10), textcoords="offset
    points", fontsize=9, color="magenta")
145 plt.annotate("0.4RDV", xy=(xl_l, yl_l), xytext=(-45, -18), textcoords="offset
    points", fontsize=9, color="magenta")
146 plt.annotate("1.6RDV", xy=(xu_r, yu_r), xytext=(8, 10), textcoords="offset
    points", fontsize=9, color="magenta")
147 plt.annotate("0.4RDV", xy=(xl_r, yl_r), xytext=(8, -18), textcoords="offset
    points", fontsize=9, color="magenta")
148
149 # Field size indication between inflection points.
150 y_field = 8
151 plt.annotate("", xy=(x_inflect_left, y_field), xytext=(x_inflect_right, y_field)
    , arrowprops=dict(arrowstyle="<->", color="black", lw=1.6))
152 plt.annotate(f"Field size = {field_size_inflection:.2f} mm", xy=(x_rdv_mid,
    y_field), xytext=(-55, -18), textcoords="offset points", fontsize=9)
153
154 # Penumbra region markers (left/right) shown only as RDV level points.
155 plt.annotate("", xy=(xu_l, yu_l), xytext=(xl_l, yl_l), arrowprops=dict(
    arrowstyle="<->", color="magenta", lw=1.5))
156 plt.annotate("", xy=(xu_r, yu_r), xytext=(xl_r, yl_r), arrowprops=dict(
    arrowstyle="<->", color="magenta", lw=1.5))
157
158 # AERB level region labels.
159 plt.annotate("90%", xy=(aerb_right[90], 90), xytext=(8, 0), textcoords="offset
    points", fontsize=8, color="green")
160 plt.annotate("75%", xy=(aerb_right[75], 75), xytext=(8, 0), textcoords="offset
    points", fontsize=8, color="green")
161 plt.annotate("60%", xy=(aerb_right[60], 60), xytext=(8, 0), textcoords="offset
    points", fontsize=8, color="green")
162
163 # Horizontal guides for AERB dose levels.
164 for lev in levels:
165     plt.axhline(lev, color="green", linestyle=":", linewidth=0.8, alpha=0.4)
166
167 info_text = (
168     f"Field size (IP-IP): {field_size_inflection:.2f} mm\n"
169     f"Symmetry: {symmetry:.2f} %\n"
170     f"90% L/R: {abs(aerb_left[90]):.2f} / {abs(aerb_right[90]):.2f} mm\n"
171     f"75% L/R: {abs(aerb_left[75]):.2f} / {abs(aerb_right[75]):.2f} mm\n"
172     f"60% L/R: {abs(aerb_left[60]):.2f} / {abs(aerb_right[60]):.2f} mm"
173 )
174
175 plt.text(
176     0.015,
177     0.985,
178     info_text,
179     transform=plt.gca().transAxes,
180     va="top",
181     ha="left",
182     fontsize=10,
183     family="DejaVu Sans",
184     bbox=dict(boxstyle="round,pad=0.45", facecolor="white", alpha=0.92,
```

```
        edgecolor="black", linewidth=0.8),
)

plt.title("Beam Profile for 6MV FFF (20x20 cm2 field)", fontsize=14)
plt.xlabel("Off Axis [mm]")
plt.ylabel("Crossline dose")
plt.savefig("Figures/Profile/6MV FFF(20 by 20)/Beam_Profile_FFF_Crossline.pdf",
    dpi=300, bbox_inches="tight")
plt.show()
```

Listing 5: Python program for electron beam profile analysis.

```
# Importing some useful library
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.interpolate import CubicSpline

import scienceplots
plt.style.use(['science', 'notebook', 'grid'])

# Data Extraction from the xlsx file
df = pd.read_excel("PROFILE Group 2.xlsx", sheet_name=3)
x = np.array(df["Off Axis [mm]"])
y = np.array(df["Inline dose"])

center_idx = np.argmin(np.abs(x))
y = y / y[center_idx] * 100

# Sort data for interpolation and robust crossing detection.
sort_idx = np.argsort(x)
x = x[sort_idx]
y = y[sort_idx]

spline = CubicSpline(x, y)
x_dense = np.linspace(x.min(), x.max(), 4000)
y_dense = spline(x_dense)


# Analysis helpers
def find_crossing_x(side_x, side_y, level):
    """Return x-position where profile crosses a given dose level."""
    for i in range(len(side_y) - 1):
        y1, y2 = side_y[i], side_y[i + 1]
        if (y1 >= level and y2 <= level) or (y1 <= level and y2 >= level):
            if y2 == y1:
                return side_x[i]
            return side_x[i] + (level - y1) * (side_x[i + 1] - side_x[i]) / (y2
                - y1)
    raise ValueError(f"No crossing found for {level}% dose level.")


# Split profile around central axis (x ~= 0)
center_idx = np.argmin(np.abs(x))

# Left side from center outward (toward negative x)
left_x = x[:center_idx + 1][::-1]
left_y = y[:center_idx + 1][::-1]

# Right side from center outward (toward positive x)
right_x = x[center_idx:]
right_y = y[center_idx:]

# Field size: lateral separation between 50% dose points.
```

```
x50_left = find_crossing_x(left_x, left_y, 50)
x50_right = find_crossing_x(right_x, right_y, 50)
FieldWidth = abs(x50_right - x50_left)

# Beam flatness: separation between 90% and 50% dose points on each side.
x90_left = find_crossing_x(left_x, left_y, 90)
x90_right = find_crossing_x(right_x, right_y, 90)
flatness_left = abs(x50_left - x90_left)
flatness_right = abs(x90_right - x50_right)
flatness_avg = 0.5 * (flatness_left + flatness_right)

# Penumbra lateral distance between 20% and 80% dose.
x80_left = find_crossing_x(left_x, left_y, 80)
x20_left = find_crossing_x(left_x, left_y, 20)
x80_right = find_crossing_x(right_x, right_y, 80)
x20_right = find_crossing_x(right_x, right_y, 20)

# Penumbra width = lateral distance between 80% and 20% dose
left_penumbra = abs(x20_left - x80_left)
right_penumbra = abs(x20_right - x80_right)

# Symmetry: maximum ratio of absorbed doses at symmetrical points more than 1 cm
    inside the 90% contour.
symmetry_half_width = min(abs(x90_left), abs(x90_right)) - 10.0
if symmetry_half_width <= 0:
    symmetry_half_width = 0.5 * min(abs(x90_left), abs(x90_right))

symmetry_x = np.linspace(0, symmetry_half_width, 600)
sym_left = np.interp(-symmetry_x, x, y)
sym_right = np.interp(symmetry_x, x, y)
sym_ratio = np.maximum(sym_left / sym_right, sym_right / sym_left)
Symmetry = float(np.max(sym_ratio))


#Plotting
fig, ax = plt.subplots(figsize=(15, 8))
ax.plot(x, y, label="Beam Profile")
ax.plot(x_dense, y_dense, color='tab:blue', linewidth=1.5, alpha=0.75)
plt.plot([x50_left, x50_right], [50, 50], color='red', linestyle='--', label='
    50% Line')
plt.plot([x50_left, x50_left], [0, 102], color='green', linestyle='--')
plt.plot([x50_right, x50_right], [0, 102], color='green', linestyle='--')
plt.plot([0, 0], [0, 105], color='blue', linestyle='--')

# Flatness markers at 90% and 50% dose points.
plt.plot([x90_left, x90_right], [90, 90], marker='o', color='orange', linestyle=
    'None')
plt.plot([x50_left, x50_right], [50, 50], marker='o', color='red', linestyle='
    None')

# Penumbra markers at 80% and 20% dose points.
plt.plot([x80_left, x80_right], [80, 80], marker='s', color='magenta', linestyle
    ='None')
plt.plot([x20_left, x20_right], [20, 20], marker='s', color='magenta', linestyle
    ='None')


# Annotations.
plt.annotate(
    f"Field size = {FieldWidth:.2f} mm",
    xy=(0.5 * (x50_left + x50_right), 50),
    xytext=(0, 18),
    textcoords='offset points',
```

```
    ha='center',
    fontsize=11,
    bbox=dict(boxstyle='round,pad=0.25', facecolor='white', alpha=0.9, edgecolor='black', linewidth=0.8),
)

info_text = (
    f"Field size: {FieldWidth:.2f} mm\n"
    f"Flatness L/R: {flatness_left:.2f} / {flatness_right:.2f} mm\n"
    f"Symmetry: {Symmetry:.3f}\n"
    f"Left penumbra: {left_penumbra:.2f} mm\n"
    f"Right penumbra: {right_penumbra:.2f} mm"
)

ax.text(0.02, 0.98, info_text, transform=ax.transAxes, va='top', ha='left',
        fontsize=11, bbox=dict(boxstyle='round,pad=0.4', facecolor='white', alpha=0.9, edgecolor='black'))
ax.set_xlabel("Distance from central axis (mm)")
ax.set_ylabel("Relative dose (%)")
plt.savefig("Figures/Profile/6MeV(10 by 10)/InlineProfile.pdf", dpi=300, bbox_inches='tight')
plt.show()
```

5 Conclusion

The measured PDD and beam-profile data for photon, FFF photon, and electron beams were consistent with the expected clinical behavior of a medical linear accelerator.

For flattened photon beams, the profile at smaller field size remained comparatively uniform in the central region, whereas the larger field size showed a clearer horn effect near the off-axis region. This is physically expected because the flattening filter is designed to compensate central-axis fluence by hardening and attenuating the beam more at the center than at the periphery. At larger field widths, this over-compensation becomes more apparent and produces peripheral dose elevation (horns). In clinical dosimetry, this confirms why profile evaluation at large fields is essential for baseline QA, monitor-unit confidence, and TPS beam-model validation.

The FFF beam profile showed a non-flat, forward-peaked distribution with a higher central axis dose and gradual off-axis fall-off. This is the characteristic signature of filter-free operation: after removing the flattening filter, there is reduced head scatter and less beam hardening from the filter, so lateral fluence compensation is absent. From a medical physics perspective, this behavior is expected and acceptable, provided machine-specific baseline quantities (inflection points, unflatness metrics, and penumbra definitions for FFF mode) are stable and reproducible for periodic QA.

The electron PDD exhibited a high dose in the near-surface to shallow-depth region, followed by a relatively rapid distal fall-off and a low-dose bremsstrahlung tail. This reflects the finite range of therapeutic electrons in water-equivalent media: electrons deposit dose efficiently up to a practical range and then lose dose sharply due to energy depletion and multiple scattering. Clinically, this supports the use of electrons for superficial targets while reducing deeper normal-tissue dose compared with photons.

Overall, the observed depth-dose and profile characteristics were in line with standard radiotherapy beam physics. The extracted dosimetric parameters (field width behavior, symmetry/flatness-type checks, penumbra trends, and depth-dose pattern) indicate that the beams are suitable for clinical implementation, subject to routine constancy checks and continued TPS-measurement agreement.

References

[1] Atomic Energy Regulatory Board (AERB), "Quality assurance in radiotherapy: Regulatory guidance and periodic qa practice," 2020.

[2] J. P. Gibbons, *Khan's The Physics of Radiation Therapy.* Philadelphia: Wolters Kluwer, 6 ed., 2020.